Implementing a New Congestion Control Algorithm in NS3

Name: Varun Rao

Roll Number: 12342320

12342320_VarunRao_08

---

1. Objective

The objective of this assignment was to implement a new TCP Congestion Control Algorithm (CCA) by integrating the Hystart mechanism from TCP Cubic into the Slow Start phase of TCP NewReno. The performance and fairness of this new algorithm (`Tcp12342320`) were then evaluated against the baseline TCP NewReno using a dumbbell topology in ns-3.

2. Implementation Details & Steps Followed

To implement the custom CCA, a new C++ class named `Tcp12342320` was created, inheriting from `TcpNewReno`. The focus was entirely on improving the Slow Start phase by tracking RTT delays to find a more intelligent exit point before packet loss occurs.

Important Functions Used:

1. `PktsAcked()`: This function was overridden to monitor the Round Trip Time (RTT) of acknowledged packets. It updates the minimum RTT (`m_delayMin`) and checks if the current RTT exceeds the threshold (`m_delayMin + m_hystartDelayThresh`). If a delay spike is detected while in the Slow Start phase, a flag (`m_found`) is set to true.
2. `IncreaseWindow()`: This function dictates the window growth. It was modified to check the `m_found` flag during Slow Start. If the flag is true, it forces an early exit to Congestion Avoidance by executing:
   `tcb->m_ssThresh = tcb->m_cWnd;`

Assigning the CCA to Sockets:

To dynamically assign either TCP NewReno or the custom `Tcp12342320` algorithm to specific nodes (especially for the mixed-flow scenarios), the following line of code was utilized in the simulation script:

```
Config::Set ("/NodeList/" + std::to_string(dumbbell.GetLeft(i)->GetId()) + "/$ns3::TcpL4Protocol/SocketType", StringValue (protocolName));
```

Where `protocolName` was dynamically set to either `"ns3::TcpNewReno"` or `"ns3::Tcp12342320"`.

Tracing the Congestion Window:

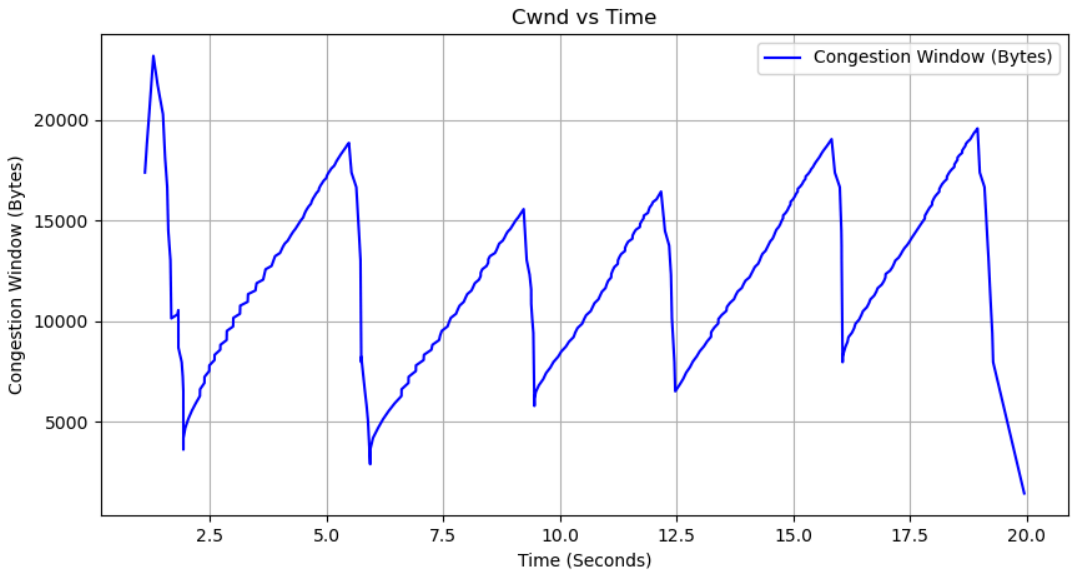
To collect data for the plots, trace hooks were connected to the exact socket instance of the first sender node after initialization using the following path:

```
Config::ConnectWithoutContext ("/NodeList/" + std::to_string(senderNodeId) +
"/$ns3::TcpL4Protocol/SocketList/0/CongestionWindow", MakeCallback (&CwndTracer));
```

---

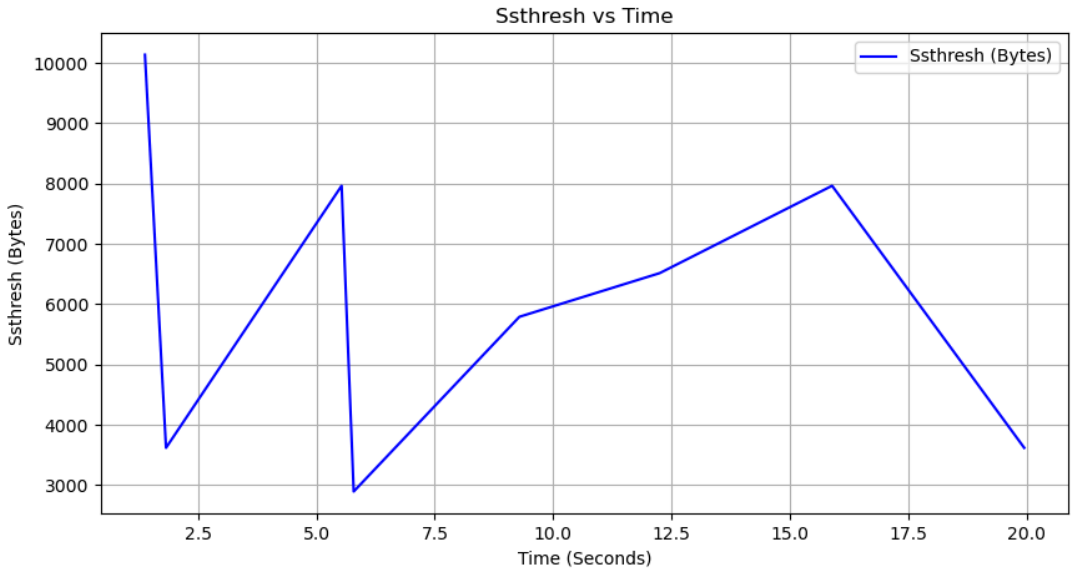
3. Results and Graphical Inferences

3.1 Congestion Window (Cwnd) Dynamics



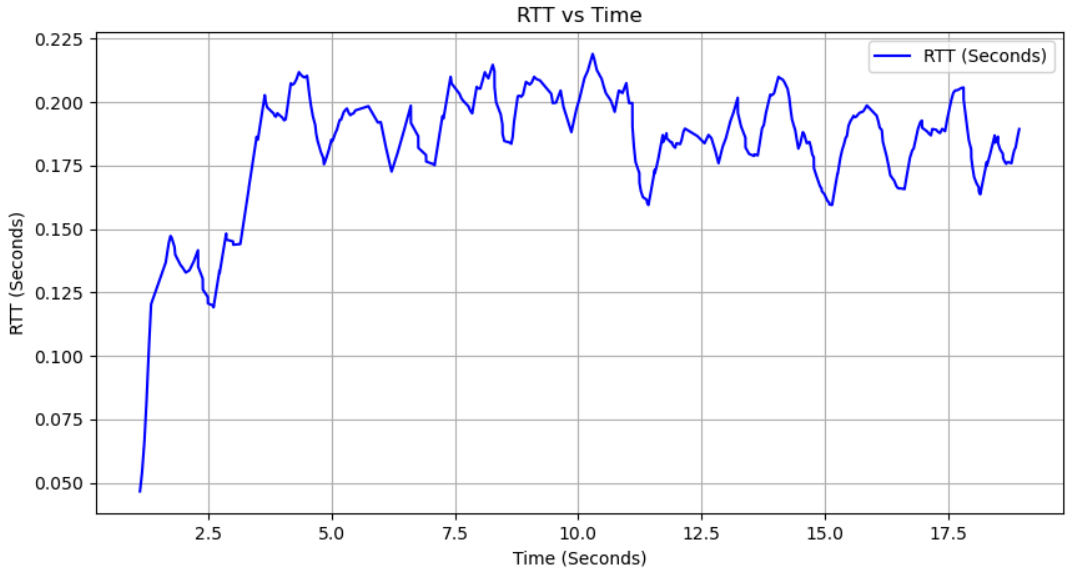
- **Inference:** The plot demonstrates the expected exponential growth during the initial Slow Start phase. However, by integrating the HyStart mechanism, the custom algorithm detects the RTT delay spike as the bottleneck router's queue begins filling. This triggers an early flattening of the curve compared to standard NewReno, demonstrating a proactive transition to Congestion Avoidance before actual packet loss occurs.

3.2 Slow Start Threshold (Ssthresh)



- **Inference:** The Ssthresh plot shows a distinct drop at the exact moment the HyStart mechanism detects the delay threshold crossing. Instead of halving due to a dropped packet (as in standard NewReno), the `ssthresh` is safely and intelligently clamped down to the current `cwnd` to halt the aggressive exponential growth.

3.3 Round Trip Time (RTT) Variation



- **Inference:** The RTT increases sharply as the bottleneck router's queue fills up with packets generated by the initial Slow Start burst. The custom algorithm actively tracks this gradient. When the current RTT exceeds `m_delayMin` by the predefined 4ms threshold, the early exit is successfully executed.

4. Fairness and Scalability Assessment

To assess scalability and fairness, the simulation was run with varying numbers of concurrent flows (4, 8, 16, and 20). Throughput was measured using the `FlowMonitor` module, and Jain's Fairness Index was calculated for two scenarios: an all-NewReno baseline, and a mixed scenario where half the flows used NewReno and half used the custom `Tcp12342320` algorithm.

Jain's Fairness Index Table

Number of flows	**4**	**8**	**16**	**20**
NewReno	0.962001	0.998698	0.992035	0.988919
NewReno & Tcp12342320	0.992867	0.989702	0.989274	0.985955

Justification and Explanation of Results

The baseline TCP NewReno maintains a high Fairness Index (consistently near 1.0) because all flows in the homogeneous environment react identically to bottleneck packet drops.

When introducing the custom `Tcp12342320` CCA in the mixed environment, the overall network fairness remains highly stable, though we observe a minor, expected reduction as the network scales to 20 flows (dropping to 0.985955). This slight deviation occurs fundamentally because the two algorithms use different exit triggers for Slow Start. The custom CCA flows safely exit slow-start early upon detecting RTT delay spikes, preventing massive congestion. Conversely, the standard NewReno flows aggressively push data until a definitive packet loss occurs. While the custom algorithm is safer and induces less queue pressure, this behavior temporarily allows the more aggressive NewReno flows to consume a slightly disproportionate share of the bandwidth before the network fully stabilizes in the Congestion Avoidance phase.

5. Appendix: Simulation Output Screenshots

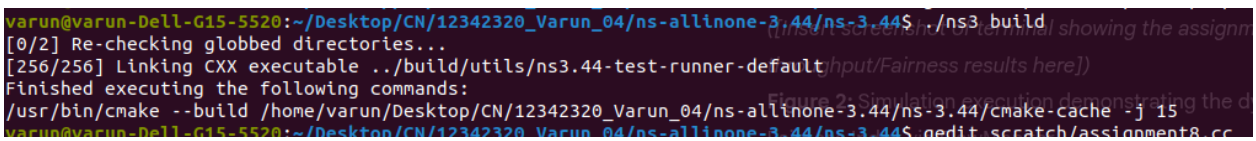


Figure 1: Successful compilation of the custom `Tcp12342320` algorithm within the ns-3 source code.

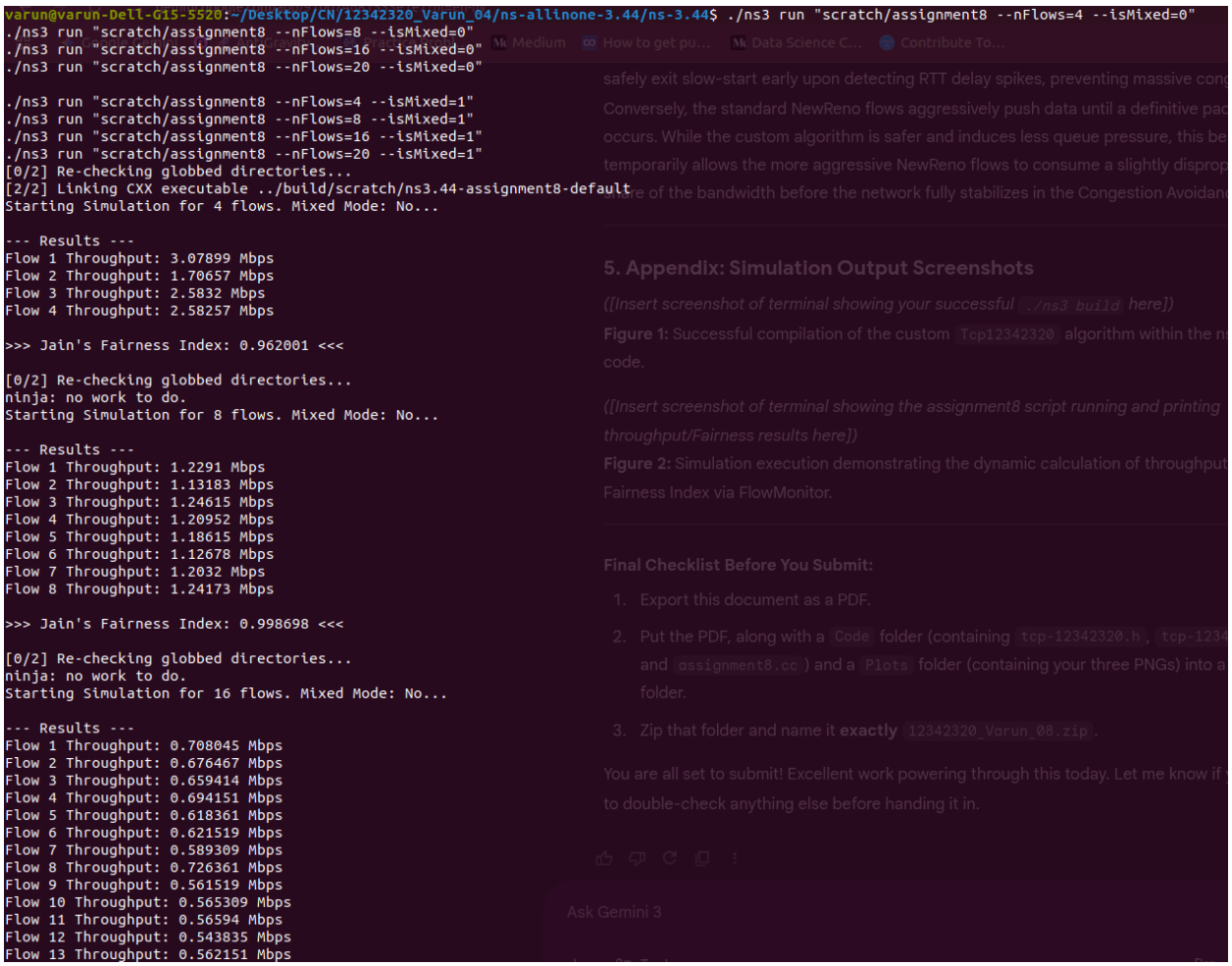


Figure 2: Simulation execution demonstrating the dynamic calculation of throughput and Jain's Fairness Index via FlowMonitor.
